AI-Driven Phishing Email Detection Using NLP

Project 2 — Final Report

Thirugnanam V S
B.Tech Electronics Engineering (VLSI Design & Technology), VIT Chennai
IICT Summer Internship on AI & Machine Learning — 2026

Mentor: Dr. Ashok, IICT
July 2026

1. Introduction

Phishing emails try to manoeuvre a recipient into clicking a link, opening an attachment or handing over credentials, and no inbox can be reviewed manually at scale. This project builds a machine learning system that classifies emails as phishing or legitimate from their text, completed as Project 2 of the IICT Summer Internship on AI and Machine Learning following the four-week workflow: data understanding and cleaning, exploratory analysis and feature engineering, model building with hyperparameter tuning, and documentation.

Every number in this report was produced by the project code in a single seeded run (random_state=42). Each value is written to a file in the outputs folder by the pipeline itself, and every figure comes from the graphs folder generated by the same run.

2. Problem Statement

Given the combined subject-and-body text of an email, predict whether it is phishing (label 1) or legitimate (label 0). The system must be built from scratch — loading, cleaning, preprocessing, feature extraction, training with tuning, and evaluation — and must compare several machine learning algorithms fairly, with special attention to recall, because a missed phishing email costs a user far more than a false alarm costs a reviewer.

3. Objectives

- Build a complete, reproducible NLP pipeline from the raw CSV to a saved, tuned model.
- Verify — not assume — how much preprocessing the pre-normalised corpus still needs, and remove measured source watermarks.
- Choose the feature representation by a pre-declared measured protocol.
- Train and tune the four models required by the project brief under identical conditions and select the best by measurement.
- Read the winning model's weights to separate genuine phishing signal from dataset artefact, and state the limitations honestly.

4. Dataset Description

The dataset is a single CSV of 82,486 emails with two columns: text_combined (subject and body merged by the dataset authors) and a binary label — 42,891 phishing and 39,595 legitimate, a comfortable 52/48 balance with no missing values. The corpus aggregates ham from Enron-era business mailboxes and public mailing lists with phishing and scam messages from public archives. Label semantics were verified by sampling rather than assumed: label-1 emails read like promotions and scams, label-0 emails like internal business mail.

5. Data Cleaning

5.1 Verify-then-apply preprocessing

The text arrives partially normalised, and the inspection code measured exactly how much: 0.0% of emails contain uppercase letters and 0.0% contain HTML remnants — but 47.2% still carry URL tokens ('http') and 94.2% contain digits. The standard pipeline (URL and HTML removal, non-letter stripping,

English stop words) therefore still earns its place, with the inspection report as evidence. One consequence recorded honestly: upstream normalisation already destroyed URL structure, so URL-based features can only be approximated at the token level.

5.2 Source watermark

The token 'enron' appears in 18.2% of legitimate emails and 0.0% of phishing ones — a label for where the ham was collected, not phishing signal — so it was added to the stop-word list, exactly as a news-agency dateline would be stripped in a fake-news task. Deeper source effects that single tokens cannot capture resurface in the Discussion.

5.3 Outliers, duplicates and conflicts

The longest raw email runs 4,279,526 characters (median: 558) — a mailing-list digest — so emails are truncated at 20,000 characters, touching 255 messages (0.31% of the data) while keeping every email's informative opening. Truncation runs before de-duplication on purpose: two long emails that differ only beyond the cap become identical afterwards, and de-duplication must operate on the text the models will actually see. A dedicated check for texts appearing under both labels found none.

Cleaning step	Rows affected	Rows remaining
Truncated over-length emails (cap 20,000 chars)	255 (modified)	82,486
Exact duplicate rows removed	409	82,077
Label-conflicting texts removed	0	82,077
Empty / digits-only emails removed	2	82,075

The final corpus holds 82,075 emails: 42,843 phishing and 39,232 legitimate (52.2% phishing), saved with three columns (text_combined, clean_text, label). The raw file is never modified.

6. Exploratory Data Analysis

Six figures were generated, each with a written observation saved to outputs/eda_summary.txt. A missing-values chart was skipped on purpose — the inspection reports show zero missing values everywhere.

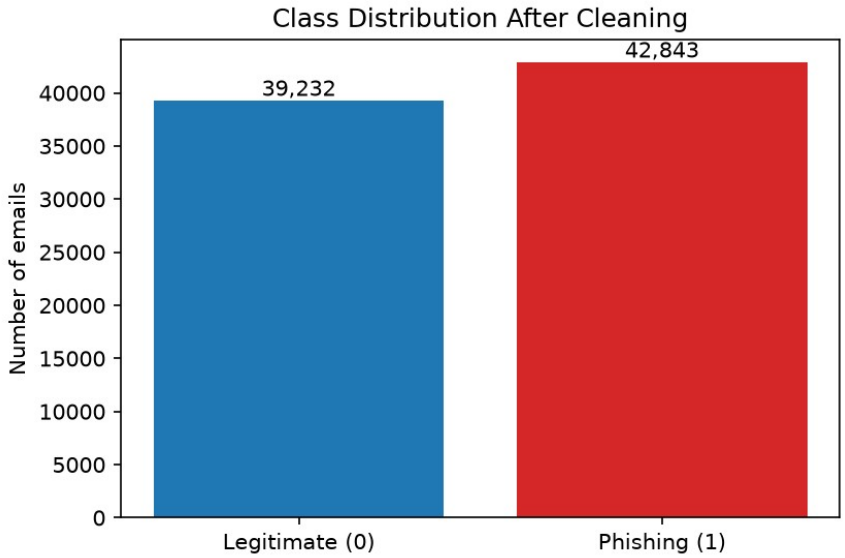

Figure 1. Class distribution after cleaning.

Figure 1 shows the near-balance after cleaning: 42,843 phishing against 39,232 legitimate. Accuracy stays a meaningful metric, no resampling is needed, and the stratified split keeps the ratio identical on both sides.

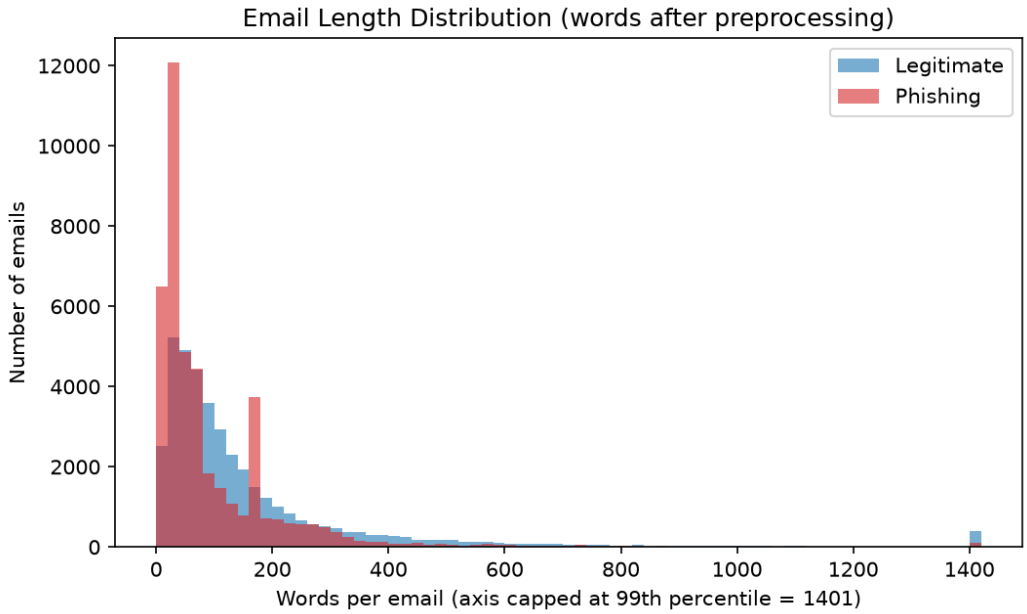

Figure 2. Email length distribution (words after preprocessing).

Legitimate emails average 168 words (median 93, standard deviation 260) while phishing emails average 103 words (median 48, standard deviation 156). Both distributions pile up on short messages and overlap heavily — length alone cannot separate the classes; the signal has to come from the words.

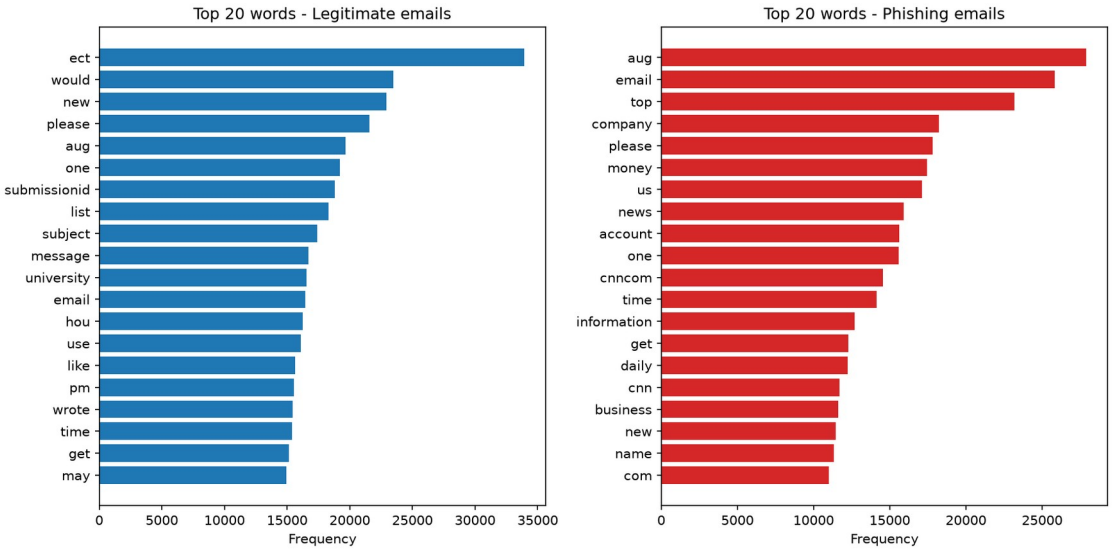

Figure 3. Twenty most frequent words per class.

The vocabularies barely overlap: only 7 of the top-20 words are shared. The legitimate side is dominated by business and scheduling vocabulary, the phishing side by promotional and action vocabulary. Two legitimate top words (ect, submissionid) are Enron system artifacts — the same source-watermark family as the 'enron' token handled in Week 1, noted here and picked up again in the Discussion.

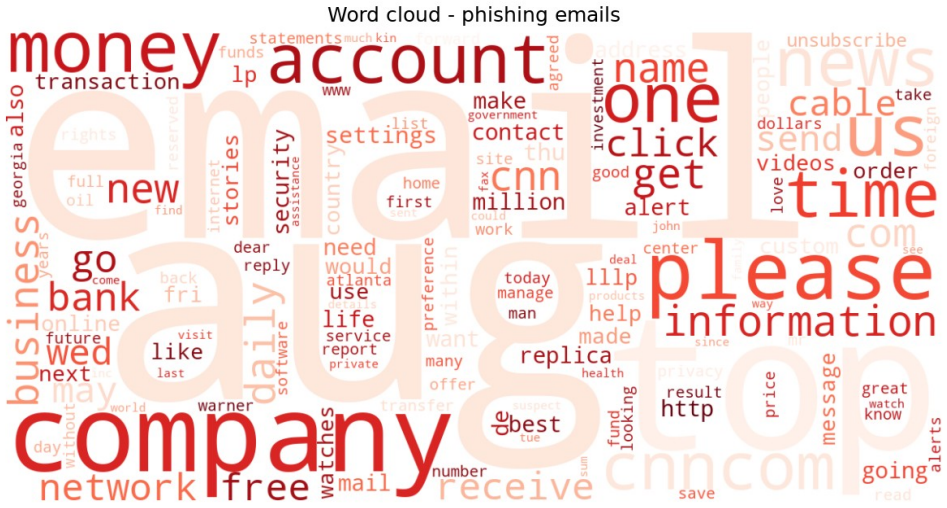

Figure 4. Word cloud — phishing emails (the legitimate-class cloud is in graphs/).

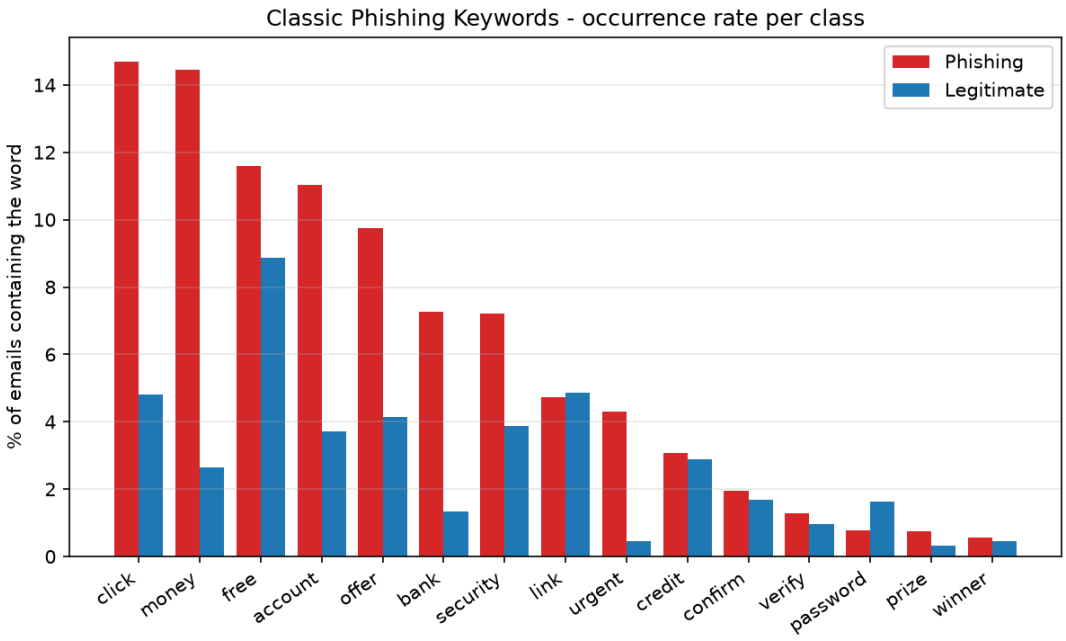

Figure 5. Classic phishing keywords — occurrence rate per class.

Figure 5 tests a list of fifteen classic scam words fixed before looking at the data. Every one occurs more often in phishing: 'click' leads at 14.7% of phishing emails versus 4.8% of legitimate, and 'money' shows the widest gap (14.4% vs 2.6%). None is remotely universal, which is exactly why the models use 5,000 word features instead of a keyword rule.

7. Feature Engineering

Order matters: the corpus was split first — 80/20, stratified, seed 42 — giving 65,660 training and 16,415 testing emails with an identical 52.2% phishing share on both sides. Every vectorizer was then fitted on the training emails only, so no vocabulary statistics leak from the test set.

CountVectorizer and TF-IDF (both at 5,000 features) were compared with the same Logistic Regression probe under 3-fold cross-validation on the training set:

Representation	CV Accuracy	CV F1 (phishing)	Mean fit time
CountVectorizer	0.9795 ± 0.0014	0.9805 ± 0.0013	0.53 s
TF-IDF	0.9792 ± 0.0010	0.9801 ± 0.0010	0.26 s

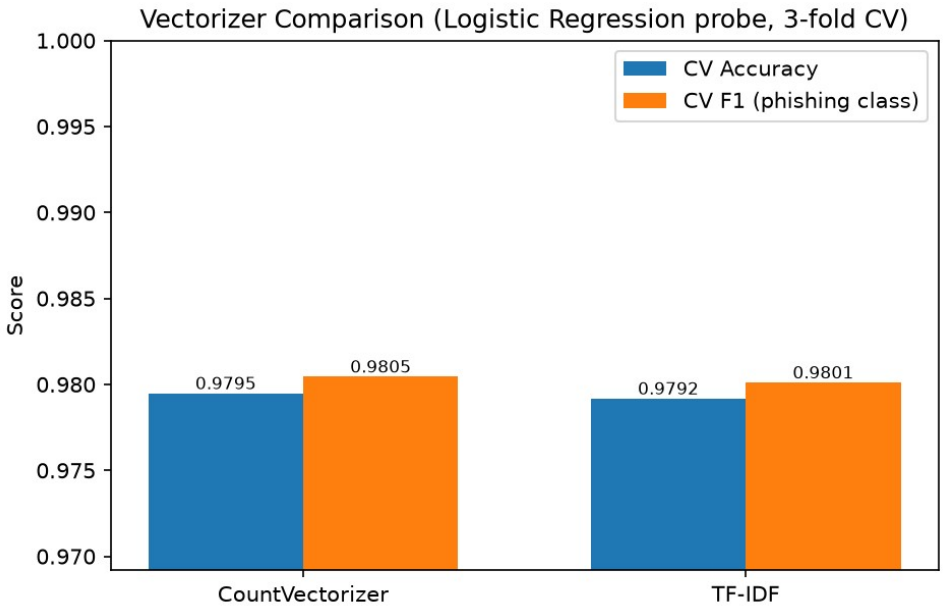

Figure 6. Vectorizer comparison (Logistic Regression probe, 3-fold CV).

The result is a statistical tie: a gap of 0.0003 in F1, inside the 0.001 tie threshold declared before the comparison was run. The pre-declared tie rule selects TF-IDF — it down-weights near-universal words and is the representation the project brief suggests. Running the comparison was still worth it: in Project 1 the identical protocol produced a decisive CountVectorizer win, so the obvious default is not always the measured winner.

8. Model Building and Hyperparameter Tuning

The four models required by the project brief were trained on identical TF-IDF matrices and scored on the identical test set:

- Logistic Regression — C grid-searched over 0.1 / 1.0 / 10.0 → best C = 10 (CV F1 0.9827, up from 0.9801 at the default)
- Multinomial Naive Bayes — alpha grid-searched over 0.1 / 0.5 / 1.0 → best alpha = 0.1 (CV F1 0.9605)
- Random Forest — 100 trees; forests are famously insensitive to adding trees beyond this point, so the budget went to the grids that move the needle
- MLP Neural Network — one hidden layer of 100 units, Adam, early stopping (patience 5), which self-regularises by construction

Grid searches ran as 3-fold cross-validation on the training set only, scored by F1 on the phishing class; the searched grids, candidates and winners are written by the pipeline to outputs/hyperparameter_tuning.csv so the scope of the search is visible rather than implied. Every stochastic model shares the project seed, all four trained models are saved to the models folder with joblib, and the fitted vectorizer is saved alongside the best model.

9. Results

Precision, recall and F1 refer to the phishing class. Full classification reports for all four models are in outputs/classification_reports.txt, with one confusion matrix per model in the graphs folder.

Model	Accuracy	Precision	Recall	F1	Train	Predict
Logistic Regression (C=10)	0.9822	0.9808	0.9851	0.9829	0.42 s	0.003 s
MLP Neural Network	0.9818	0.9803	0.9849	0.9826	46.6 s	0.043 s
Random Forest	0.9814	0.9850	0.9792	0.9821	14.3 s	0.085 s
Multinomial NB (alpha=0.1)	0.9567	0.9743	0.9419	0.9578	0.02 s	0.005 s

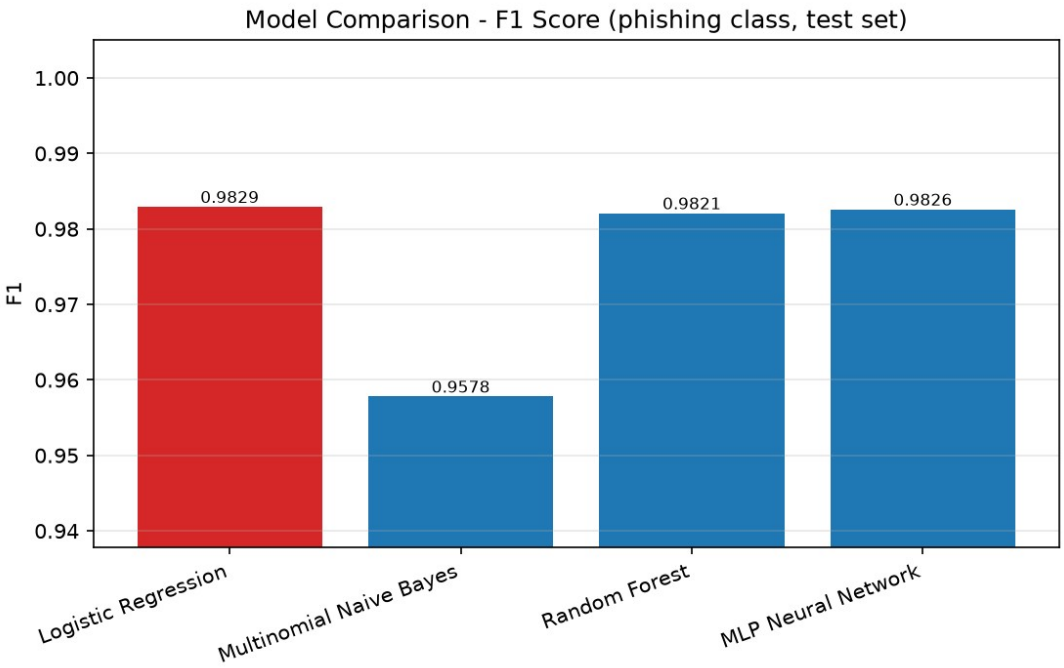

Figure 7. F1 score of the four models on the held-out test set (best model highlighted).

Tuned Logistic Regression is the best model with 98.22% accuracy, F1 0.9829 and — decisively for this domain — the highest recall (0.9851). Its confusion matrix (Figure 8) shows 293 errors out of 16,415 test emails: 165 legitimate emails flagged as phishing and 128 phishing emails missed, about 15 misses per 1,000 phishing emails.

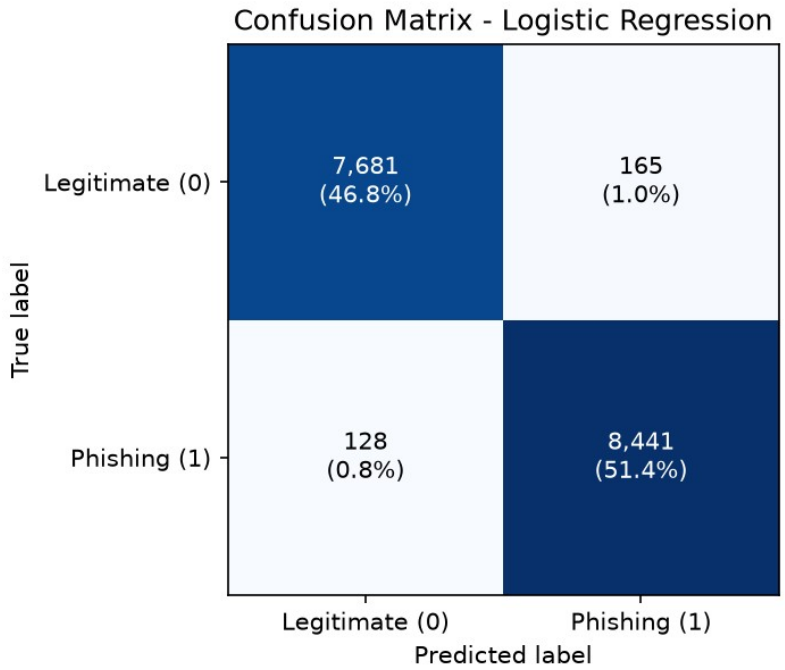

Figure 8. Confusion matrix of the best model (tuned Logistic Regression).

The race at the top is extraordinarily tight: the MLP (0.9826) and the Random Forest (0.9821) sit within 0.0008 F1 of the winner. With vocabularies this disjoint, the classes are already close to linearly separable, so the MLP's extra capacity mostly re-draws the same boundary at 46.6 seconds of training against 0.42, while the forest buys the best precision of the four (0.9850) at the cost of the lowest recall among the top three (0.9792). Naive Bayes trails at 0.9578 despite tuned smoothing; its 498 missed phishing emails — nearly four times the winner's — make it the wrong pick here even where its speed appeals. Tuning earned its keep twice: alpha=0.1 beat the Naive Bayes default, and C=10 lifted the Logistic Regression probe's CV F1 from 0.9801 to 0.9827.

10. Discussion

10.1 Recall first

The two error types are not symmetric: a false positive sends a legitimate email to a review folder, a false negative delivers a scam. The tuned Logistic Regression tops both F1 and recall, so it is the right pick twice over. Its 128 misses are informative as a group: missed phishing emails average 180 words (median 70) against 104 words for phishing overall — the misses skew towards longer messages that mix in enough neutral business vocabulary to blur the boundary, while short, punchy scams are caught almost perfectly. Misclassified emails of both kinds run longer than correctly classified ones on average (164 vs 133 words).

10.2 What the model actually learned

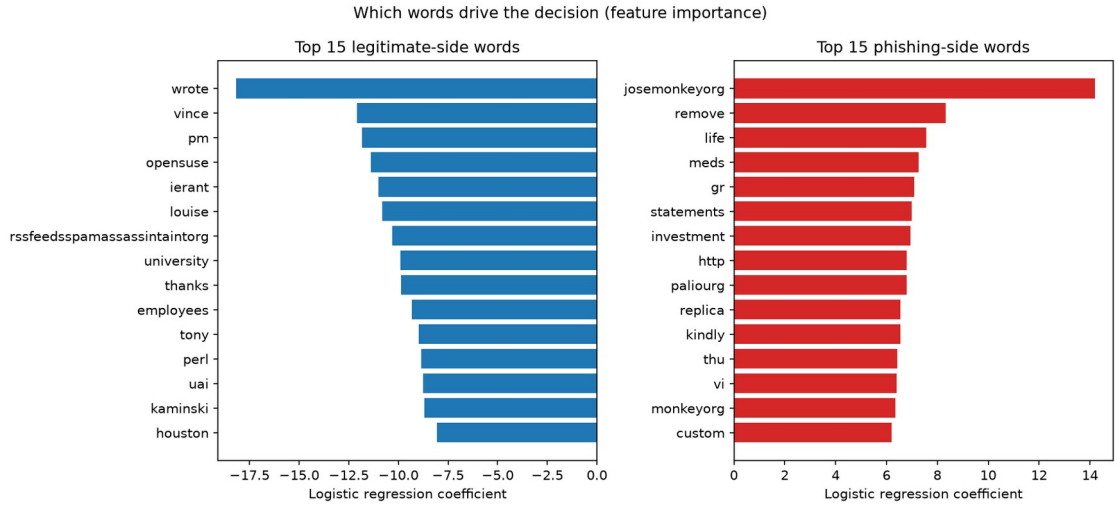

Figure 9. The fifteen strongest words per side, from the winning model's coefficients.

Reading the strongest coefficients (Figure 9) is sobering in the most useful way. The phishing side contains genuine scam vocabulary — meds, replica, investment, kindly, remove, and the URL token http — but the single strongest feature is 'josemonkeyorg', the hostname of a public phishing archive, joined by 'paliourg', a token from the Enron-Spam benchmark's collection process. The legitimate side mixes real business signal (wrote, thanks, employees) with the first names of Enron executives (vince, louise), a mailing-list token (opensuse) and, most tellingly, 'rssfeedsspamassassintaintorg' — a SpamAssassin collection artefact. After removing the one watermark nameable in advance, the model found the rest by itself: it is partly a phishing detector and partly an archive detector, and near-ceiling scores on combined corpora should be described in exactly those terms.

10.3 Limitations

The scores measure in-distribution separation of the source archives. A mailbox whose legitimate traffic does not look like Enron-era business mail — or whose phishing does not look like archived scams — would need retraining and fresh evaluation. Features are unigram TF-IDF only; URL structure was destroyed upstream; and the labels are inherited from the archives rather than from per-message review.

11. Conclusion

A complete phishing detection pipeline was built from scratch with a measurement behind every decision: the pre-normalisation claim was verified (0.0% uppercase, 47.2% URL tokens), a quantified watermark was removed (18.2% vs 0.0%), 255 outliers were truncated before de-duplication, the representation was chosen by a pre-declared tie protocol, and the compute-cheap hyperparameters were grid-searched. Tuned Logistic Regression won with 98.22% accuracy, F1 0.9829 and the best recall (0.9851, 128 misses in 8,569 phishing emails), with the MLP and Random Forest within 0.0008 F1 behind. Coefficient inspection showed the model splits its attention between genuine scam vocabulary and the names of the archives the corpus was assembled from — an honest description of what near-perfect accuracy means here.

12. Future Scope

- Cross-source evaluation: train and test on different archives to measure the archive-recognition share of the score.
- Richer URL and header features on a corpus that preserves them.
- Calibrated probability outputs so borderline emails can be routed to human review.
- A small fine-tuned transformer baseline under the same leakage-controlled protocol.
- A simple demo application (Flask or Streamlit) around the saved model and vectorizer.

13. References

[1] I. Fette, N. Sadeh, and A. Tomasic, “Learning to detect phishing emails,” in Proc. 16th Int. Conf. on World Wide Web (WWW), 2007, pp. 649–656.

[2] A. Almomani, B. B. Gupta, S. Atawneh, A. Meulenberg, and E. Almomani, “A survey of phishing email filtering techniques,” IEEE Communications Surveys & Tutorials, vol. 15, no. 4, pp. 2070–2090, 2013.

[3] B. Klimt and Y. Yang, “The Enron corpus: A new dataset for email classification research,” in Proc. ECML, LNCS vol. 3201, Springer, 2004, pp. 217–226.

[4] V. Metsis, I. Androutsopoulos, and G. Paliouras, “Spam filtering with naive Bayes — which naive Bayes?” in Proc. 3rd Conf. on Email and Anti-Spam (CEAS), 2006.

[5] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

14. Appendix — Project Structure and Reproduction

The project is a modular Python package driven by one entry point. Placing phishing_email.csv in dataset/raw/ and running python main.py reproduces every number and figure in this report in roughly 10–15 minutes; all randomness is fixed at seed 42.

- src/config.py — paths, labels, truncation cap, seed and settings
- src/data_loader.py — load and validate the CSV
- src/data_inspection.py — before/after evidence reports incl. marker rates
- src/data_cleaning.py — truncation, duplicates, label-conflict check
- src/text_preprocessing.py — normalisation pipeline (enron as stop word)
- src/eda.py — the six figures plus written observations
- src/feature_engineering.py — stratified split, vectorizer comparison, features
- src/model_training.py — the four models, grid-search tuning, training and saving
- src/model_evaluation.py — metrics, confusion matrices, comparison, best-model analysis, feature importance